

jQuery

Practicals with Answers

[JavaScript Library](#) | [DOM](#) | [AJAX](#) | [Events](#)

[Selectors & DOM Manipulation](#) · [Events & Effects](#) · [AJAX with jQuery](#) · [DOM Traversal & Manipulation](#) · [jQuery Plugins & UI Enhancements](#)

Practical 1: Selectors & DOM Manipulation

Aim: Use jQuery selectors and methods to select and manipulate DOM elements.

Code:

```
<!DOCTYPE html>
<html>
<head>
  <script src="https://code.jquery.com/jquery-3.7.0.min.js"></script>
</head>
<body>
  <h1 id="title">jQuery Demo</h1>
  <p class="info">Paragraph One</p>
  <p class="info">Paragraph Two</p>
  <ul id="list">
    <li>Item A</li><li>Item B</li><li>Item C</li>
  </ul>
  <button id="changeBtn">Change Content</button>

  <script>
    $(document).ready(function() {
      // Basic selectors (same as CSS)
      console.log($("#title").text());    // "jQuery Demo"
      console.log($(".info").length);    // 2
      console.log($("li").first().text()); // "Item A"
      console.log($("li:last").text());  // "Item C"

      // Modify content & attributes
      $("#changeBtn").click(function() {
        $("#title").text("Title Changed!");
        $(".info").css({ color: "blue", fontWeight: "bold" });
        $(".info:first").addClass("highlight").css("background", "yellow");
        $("#list li").each(function(i) {
          $(this).text("Item " + (i + 1));
        });
      });

      // HTML, val, attr
      $("li:first").html("<b>Bold Item</b>");
      $("#title").attr("data-lang", "en");
    });
  </script>
</body>
</html>
```

Explanation:

\$(selector): select elements. **\$(document).ready(fn)**: run after DOM loads. **.text()**: get/set text. **.html()**: get/set HTML. **.css()**: get/set styles. **.addClass/removeClass/toggleClass**: class manipulation. **.attr()**: get/set attributes. **.each(fn)**: iterate matched elements. **\$(this)**: current element in callback.

Output: Button click changes title, makes paragraphs blue/bold, highlights first, rennumbers list items. First list item becomes bold HTML.

Practical 2: Events & Effects

Aim: Attach events and apply jQuery built-in effects and animations.

Code:

```
<!DOCTYPE html>
<html>
<head>
  <script src="https://code.jquery.com/jquery-3.7.0.min.js"></script>
  <style>
    .box { width:150px; height:80px; background:#3b82f6;
      color:white; display:flex; align-items:center;
      justify-content:center; border-radius:8px; margin:10px; }
    .panel { background:#dbeafe; padding:15px; border-radius:8px;
      margin:10px 0; display:none; }
  </style>
```

```

</head>
<body>
  <div class="box" id="box">Click Me</div>
  <div class="panel" id="panel">Hidden Panel Content</div>
  <button id="slideBtn">Toggle Slide</button>
  <button id="fadeBtn">Fade Toggle</button>
  <button id="animBtn">Animate</button>

  <script>
    $(document).ready(function() {
      // Click event
      $("#box").click(function() {
        $(this).toggleClass("active")
          .css("background", $(this).hasClass("active")?"#16a34a":"#3b82f6");
      });

      // Hover event
      $("#box").hover(
        function() { $(this).fadeOut(200).fadeIn(200); },
        function() { $(this).css("opacity", 1); }
      );

      // Show/Hide effects
      $("#slideBtn").click(()=> $("#panel").slideToggle(400));
      $("#fadeBtn").click(()=> $("#panel").fadeToggle(600));

      // Custom animation
      $("#animBtn").click(function() {
        $("#box").animate({
          width: "250px",
          opacity: 0.7,
          marginLeft: "50px"
        }, 500, function() {
          $(this).animate({ width:"150px", opacity:1, marginLeft:"10px" }, 500);
        });
      });

      // Event delegation (for dynamic elements)
      $(document).on("click", ".dynamic-item", function() {
        alert("Dynamic: " + $(this).text());
      });
    });
  </script>
</body>
</html>

```

Explanation:

.click(fn): click handler. **.hover(enter,leave):** mouseover/out. **.fadeIn/Out/Toggle:** opacity animation. **.slideUp/Down/Toggle:** height animation. **.animate({props}, duration, callback):** custom property animation. **.toggleClass:** add/remove class. **\$(document).on('event','selector',fn):** event delegation for dynamic elements.

Output: Box toggles green/blue on click. Panel slides/fades on button clicks. Animate button stretches box right then returns. Hover causes quick flash.

Practical 3: AJAX with jQuery

Aim: Make AJAX GET and POST requests using jQuery's \$.ajax, \$.get, \$.post.

Code:

```

<script src="https://code.jquery.com/jquery-3.7.0.min.js"></script>
<div id="output"></div>
<input id="userId" type="number" value="1" min="1" max="10">
<button id="loadBtn">Load User</button>
<button id="postBtn">Post Data</button>

<script>
$(document).ready(function() {
  // $.get – shorthand GET request

```

```

$("#loadBtn").click(function() {
    const id = $("#userId").val();
    $("#output").html("<i>Loading...</i>");

    $.get(`https://jsonplaceholder.typicode.com/users/${id}`,
        function(data) {
            $("#output").html(`
                <b>Name:</b>  ${data.name}<br>
                <b>Email:</b>  ${data.email}<br>
                <b>City:</b>   ${data.address.city}
            `);
        }
    );
});

// $.post – shorthand POST request
$("#postBtn").click(function() {
    $.post("https://jsonplaceholder.typicode.com/posts",
        { title: "jQuery Post", body: "Test body", userId: 1 },
        function(data) {
            $("#output").html(`
                <b>Created ID:</b>  ${data.id}<br>
                <b>Title:</b>  ${data.title}
            `);
        }
    );
});

// Full $.ajax with all options
$.ajax({
    url:      "https://jsonplaceholder.typicode.com/posts/1",
    method:   "GET",
    dataType: "json",
    success:  data => console.log("Post title:", data.title),
    error:    err  => console.error("Failed:", err),
    complete: ()   => console.log("Request finished")
});
});
</script>

```

Explanation:

\$.get(url, callback): shorthand GET. **\$.post(url, data, callback):** shorthand POST. **\$.ajax({options}):** full-control AJAX with method, dataType, success, error, complete callbacks. **dataType:'json':** auto-parse response. **beforeSend:** runs before request (e.g., show spinner). jQuery automatically serializes JS objects to query strings or JSON for requests.

Output: Load User button fetches and displays name/email/city for selected user ID. Post button creates a post and shows new ID. \$.ajax logs post title.

Practical 4: DOM Traversal & Manipulation

Aim: Navigate the DOM tree and manipulate elements using jQuery traversal methods.

Code:

```

<!DOCTYPE html>
<html>
<head>
    <script src="https://code.jquery.com/jquery-3.7.0.min.js"></script>
</head>
<body>
    <div class="container">
        <div class="parent">
            <p class="child">Child 1</p>
            <p class="child active">Child 2 (active)</p>
            <p class="child">Child 3</p>
        </div>
    </div>
    <table id="myTable" border="1">

```

```

<tr><th>Name</th><th>Score</th><th>Action</th></tr>
<tr><td>Alice</td><td>92</td><td><button class="del">Delete</button></td></tr>
<tr><td>Bob</td><td>87</td><td><button class="del">Delete</button></td></tr>
<tr><td>Carol</td><td>95</td><td><button class="del">Delete</button></td></tr>
</table>
<button id="addRow">Add Row</button>

<script>
$(document).ready(function() {
  // Traversal
  $(".active").parent().css("border", "2px solid blue");
  $(".active").siblings().css("color", "gray");
  $(".active").prev().css("font-weight", "bold");
  $(".active").next().css("font-style", "italic");
  $(".parent").children(".child").first().css("color", "green");

  // Find (search within)
  $(".container").find("p").last().css("background", "#fef9c3");

  // Delete row on button click (event delegation)
  $(document).on("click", ".del", function() {
    $(this).closest("tr").fadeOut(300, function() { $(this).remove(); });
  });

  // Add new row
  let count = 4;
  $("#addRow").click(function() {
    $("#myTable").append(
      `<tr><td>Student ${count}</td><td>${70+count}</td>
      <td><button class="del">Delete</button></td></tr>`
    );
    count++;
  });
});
</script>
</body>
</html>

```

Explanation:

Traversal: **.parent()** up one level. **.children()** direct children. **.siblings()** all siblings. **.prev()/next()** adjacent siblings. **.find(selector)** search descendants. **.closest(selector)** nearest matching ancestor. **.first()/last()/eq(n)** filter by position. **.append(html)** add at end. **.prepend()** add at start. **.remove()** delete element.

Output: Parent gets blue border, siblings turn gray. Delete button fades out its row. Add Row appends new table row with a working delete button via delegation.

Practical 5: jQuery Plugins & UI Enhancements

Aim: Use jQuery plugins and create interactive UI features.

Code:

```

<!DOCTYPE html>
<html>
<head>
  <script src="https://code.jquery.com/jquery-3.7.0.min.js"></script>
  <style>
    .accordion-header { background:#1e3a5f; color:white; padding:10px 15px;
      cursor:pointer; border-radius:4px; margin-bottom:2px; user-select:none; }
    .accordion-body { background:#f0f4ff; padding:12px; display:none;
      border-radius:0 0 4px 4px; margin-bottom:4px; }
    .tab-btn { padding:8px 16px; cursor:pointer; border:1px solid #cbd5e1;
      background:white; border-radius:4px 4px 0 0; margin-right:2px; }
    .tab-btn.active { background:#1e3a5f; color:white; border-color:#1e3a5f; }
    .tab-pane { display:none; padding:12px; border:1px solid #cbd5e1; }
    .tab-pane.active { display:block; }
    #charCount { font-size:12px; color:#64748b; }
  </style>
</head>

```

```

<body style="padding:16px;font-family:sans-serif">
  <!-- Custom Accordion -->
  <div class="accordion-header">+ What is jQuery?</div>
  <div class="accordion-body">jQuery is a fast JS library simplifying DOM manipulation, events, and AJAX.</div>
  <div class="accordion-header">+ Why use jQuery?</div>
  <div class="accordion-body">Cross-browser compatibility, short syntax, huge plugin ecosystem.</div>

  <!-- Custom Tabs -->
  <div style="margin-top:16px">
    <button class="tab-btn active" data-tab="tab1">HTML</button>
    <button class="tab-btn" data-tab="tab2">CSS</button>
    <button class="tab-btn" data-tab="tab3">JS</button>
  </div>
  <div class="tab-pane active" id="tab1">HTML structures web content.</div>
  <div class="tab-pane" id="tab2">CSS styles and layouts pages.</div>
  <div class="tab-pane" id="tab3">JS adds interactivity and logic.</div>

  <!-- Live char counter -->
  <textarea id="bio" placeholder="Write bio..." maxlength="100"
    style="width:100%;margin-top:12px;padding:8px;border:1px solid #cbd5e1;
      border-radius:6px;resize:none" rows="3"></textarea>
  <div id="charCount">0 / 100 characters</div>

  <script>
  $(document).ready(function() {
    // Accordion
    $(".accordion-header").click(function() {
      $(this).next(".accordion-body").slideToggle(250);
    });

    // Tabs
    $(".tab-btn").click(function() {
      const target = $(this).data("tab");
      $(".tab-btn, .tab-pane").removeClass("active");
      $(this).addClass("active");
      $("##" + target).addClass("active");
    });

    // Live character counter
    $("#bio").on("input", function() {
      const len = $(this).val().length;
      $("#charCount").text(`${len} / 100 characters`)
        .css("color", len > 80 ? "red" : "#64748b");
    });
  });
</script>
</body>
</html>

```

Explanation:

.data('attr'): read data-* HTML attributes. **.slideToggle()**: accordion animation. Pattern for tabs: remove .active from all, add to clicked tab and matching pane. **.on('input', fn)**: fires on every keystroke. **.val().length**: current text length. Custom plugins follow: **\$.fn.pluginName = function(options){...}** pattern to extend jQuery.

Output: Accordion headers expand/collapse on click. Tab buttons switch content panels. Textarea shows live character count, turns red above 80 chars.

-- End of Practicals --